

01_profiling_fuentes_validacion_join

May 9, 2026

1 01. Profiling de Fuentes y Validacion de Join

1.1 Goal

Load the two Excel files, profile the schema and quality, and document why a direct join by company name is not reliable.

1.2 Inputs

- leads.xlsx (esperado en la raíz del proyecto; fallback: 01_data_ingestion_enrichment/leads.xlsx)
- proyectos_empresa.xlsx (esperado en la raíz del proyecto; fallback: 01_data_ingestion_enrichment/proyectos_empresa.xlsx)

1.3 Outputs (artefactos de evidencia en 01_data_ingestion_enrichment/outputs/)

- profiling_leads_columns.csv
- profiling_horas_columns.csv
- profiling_join_company_match_summary.csv
- profiling_exact_matches_norm.csv (puede quedar vacío si no hay matches)
- profiling_fuzzy_best_matches_top200.csv
- profiling_fuzzy_matches_score_ge_80.csv

```
[2]: from pathlib import Path
from typing import Iterable, Optional

import pandas as pd

def find_project_root(start: Optional[Path] = None) -> Path:
    """Encuentra la raíz del proyecto sin depender frágilmente del cwd.

    Criterio: la raíz contiene las carpetas `01_data_ingestion_enrichment/` y
    ↪ `02_data_cleaning/`.
    """
    start = (start or Path.cwd()).resolve()
    for p in [start, *start.parents]:
        if (p / "01_data_ingestion_enrichment").is_dir() and (p /
    ↪ "02_data_cleaning").is_dir():
            return p
```

```

    raise FileNotFoundError(
        "No se pudo localizar la raíz del proyecto.\n"
        "Sugerencia: abre la carpeta raíz del repo en VS Code y vuelve a
↪ejecutar.\n"
        f"Directorio actual (cwd): {start}"
    )

def pick_existing(candidates: Iterable[Path], *, label: str) -> Path:
    """Devuelve el primer path existente; si ninguno existe, falla con mensaje
↪claro."""
    candidates = [Path(p) for p in candidates]
    for p in candidates:
        if p.exists():
            return p
    tried = "\n".join([f" - {p.resolve()}" for p in candidates])
    raise FileNotFoundError(f"No se encontró {label}. Rutas probadas:\n{tried}")

def ensure_dir(dir_path: Path) -> Path:
    dir_path = Path(dir_path)
    dir_path.mkdir(parents=True, exist_ok=True)
    return dir_path

def save_df_csv(
    df: pd.DataFrame,
    out_path: Path,
    *,
    index: bool = False,
    encoding: str = "utf-8-sig",
) -> Path:
    """Guarda un DataFrame a CSV asegurando carpeta e imprimiendo evidencia."""
    out_path = Path(out_path)
    ensure_dir(out_path.parent)
    df.to_csv(out_path, index=index, encoding=encoding)
    print(f"[OK] Guardado: {out_path.resolve()}")
    print(f"      shape: {df.shape}")
    return out_path

ROOT_DIR = find_project_root()
INGESTION_DIR = ROOT_DIR / "01_data_ingestion_enrichment"
OUTPUT_DIR = ensure_dir(INGESTION_DIR / "outputs")

# Inputs esperados: preferir raíz del proyecto; fallback a
↪01_data_ingestion_enrichment/
LEADS_FILE = pick_existing(
    [ROOT_DIR / "leads.xlsx", INGESTION_DIR / "leads.xlsx"],
    label="leads.xlsx (input)",
)

```

```

HORAS_FILE = pick_existing(
    [ROOT_DIR / "proyectos_empresa.xlsx", INGESTION_DIR / "proyectos_empresa.
    ↪xlsx"],
    label="proyectos_empresa.xlsx (input)",
)

print("ROOT_DIR      ->", ROOT_DIR)
print("INGESTION_DIR ->", INGESTION_DIR)
print("OUTPUT_DIR     ->", OUTPUT_DIR.resolve())
print("LEADS_FILE      ->", LEADS_FILE.resolve())
print("HORAS_FILE      ->", HORAS_FILE.resolve())

```

```

ROOT_DIR      -> E:\TESIS MAESTRIA\Desarrollo_clustering_maestria
INGESTION_DIR -> E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment
OUTPUT_DIR     -> E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs
LEADS_FILE      -> E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\leads.xlsx
HORAS_FILE      -> E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\proyectos_empresa.xlsx

```

1.4 Nota de diseño (standalone + contrato de IO)

- Este notebook es **standalone**: lee `leads.xlsx` y `proyectos_empresa.xlsx` desde la raíz del proyecto (con fallback) y escribe únicamente en `01_data_ingestion_enrichment/outputs/`.
- No asume variables en memoria de otros notebooks. Su propósito es generar **evidencia/profiling** para decidir si un join por nombre de empresa es viable.
- Outputs esperados en `01_data_ingestion_enrichment/outputs/`:
 - `profiling_leads_columns.csv`
 - `profiling_horas_columns.csv`
 - `profiling_join_company_match_summary.csv`
 - `profiling_exact_matches_norm.csv`
 - `profiling_fuzzy_best_matches_top200.csv`
 - `profiling_fuzzy_matches_score_ge_80.csv`

Nota: el staging de limpieza ([02_data_cleaning/01_staging_empresas_normalizadas.ipynb](#)) **no consume** estos outputs; son artefactos de auditoría y diagnóstico.

1.5 1. Imports y configuración

```

[3]: import pandas as pd
import numpy as np
import re
import unicodedata
from pathlib import Path

try:
    from IPython.display import display # Jupyter / VS Code Notebooks

```

```

except Exception:
    def display(x): # fallback minimal
        print(x)

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 200)

```

1.6 2. Configuración de archivos

```

[4]: # Configuración de ejecución
# - Usa N_FILAS_PRUEBA para iterar rápido (None = dataset completo).
N_FILAS_PRUEBA = None # p.ej. 500 para prueba rápida

print("N_FILAS_PRUEBA ->", N_FILAS_PRUEBA)
print("Inputs:")
print(" -", LEADS_FILE.resolve())
print(" -", HORAS_FILE.resolve())
print("Outputs dir:")
print(" -", OUTPUT_DIR.resolve())

```

N_FILAS_PRUEBA -> None

Inputs:

- E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\leads.xlsx
- E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\proyectos_empresa.xlsx

Outputs dir:

- E:\TESIS

MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs

1.7 3. Función de carga de datos

```

[5]: def leer_archivo(path, nrows=None, **kwargs):
    """Lee CSV o Excel con validación de existencia y errores explicativos."""
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(f"No existe el archivo: {path.resolve()}")

    suffix = path.suffix.lower()
    if suffix == ".csv":
        return pd.read_csv(path, nrows=nrows, **kwargs)
    if suffix in (".xlsx", ".xls"):
        try:
            return pd.read_excel(path, nrows=nrows, **kwargs)
        except ImportError as e:
            raise ImportError(
                "No puedo leer .xlsx porque falta un motor (normalmente_
↪`openpyxl`). "
                "Instala con: pip install openpyxl"
            )

```

```

    ) from e

    raise ValueError(f"Formato no soportado: {suffix} ({path.name})")

```

1.8 4. Carga de datasets

```

[6]: df_leads = leer_archivo(LEADS_FILE, nrow=N_FILAS_PRUEBA)
df_horas = leer_archivo(HORAS_FILE, nrow=N_FILAS_PRUEBA)

print("[INPUT] Leads:", df_leads.shape, "|", LEADS_FILE.resolve())
print("[INPUT] Horas:", df_horas.shape, "|", HORAS_FILE.resolve())

[INPUT] Leads: (440, 15) | E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\leads.xlsx
[INPUT] Horas: (412, 18) | E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\proyectos_empresa.xlsx

e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
    warn("Workbook contains no default style, apply openpyxl's default")

```

1.9 4.b Perfilado rápido (calidad y esquema)

```

[7]: def _profile_table(df: pd.DataFrame) -> pd.DataFrame:
    out = pd.DataFrame({
        "columna": df.columns,
        "dtype": [str(df[c].dtype) for c in df.columns],
        "n_null": [int(df[c].isna().sum()) for c in df.columns],
        "pct_null": [float(df[c].isna().mean() * 100) for c in df.columns],
        "n_unique": [int(df[c].nunique(dropna=True)) for c in df.columns],
    })
    return out.sort_values(["pct_null", "n_unique"], ascending=[False, True]).
    ↪reset_index(drop=True)

def _normalize_for_compare(value) -> str:
    if value is None:
        return ""
    if isinstance(value, float) and np.isnan(value):
        return ""
    s = str(value).strip()
    if not s:
        return ""
    s = unicodedata.normalize("NFKD", s)
    s = "".join(ch for ch in s if not unicodedata.combining(ch))
    s = s.upper()
    s = re.sub(r"[^A-Z0-9 ]+", " ", s)
    s = re.sub(r"\s+", " ", s).strip()

```

```

return s

def _company_quality(df: pd.DataFrame, col: str, label: str) -> None:
    if col not in df.columns:
        print(f"[{label}] Columna no encontrada: {col}")
        return
    s = df[col]
    s_str = s.astype("string")
    blank = s_str.isna() | (s_str.str.strip() == "")
    print(f"\n[{label}] Calidad de {col}:")
    print("- filas:", len(df))
    print("- n_null:", int(s.isna().sum()))
    print("- n_blank (null o vacío):", int(blank.sum()))
    print("- n_unique (no-null):", int(s.nunique(dropna=True)))
    print("- n_duplicadas (por valor no-null):", int(s.dropna().duplicated().
↪sum()))

    top = (
        s_str.fillna("")
        .map(lambda x: x.strip())
        .replace("", pd.NA)
        .dropna()
        .value_counts()
        .head(15)
        .reset_index()
    )
    if len(top):
        top.columns = [col, "freq"]
        display(top)

    # Normalización rápida para estimar colisiones
    norm = s_str.map(_normalize_for_compare)
    norm = norm.replace("", pd.NA).dropna()
    if len(norm):
        print("- n_unique_norm:", int(norm.nunique()))
        print("- colisiones_por_norm (valores distintos que caen al mismo norm):
↪", int(norm.duplicated().sum()))

def _exact_match_summary() -> dict:
    """Resumen cuantitativo de coincidencias exactas Company vs EMPRESA."""
    if "Company" not in df_leads.columns or "EMPRESA" not in df_horas.columns:
        print("\n[JOIN] No se puede calcular resumen: falta Company o EMPRESA.")
        return {}

    leads_raw = (df_leads["Company"].astype("string").fillna("").map(lambda x:
↪x.strip()))

```

```

    horas_raw = (df_horas["EMPRESA"].astype("string").fillna(" ").map(lambda x:
↪x.strip()))
    leads_raw = leads_raw.replace("", pd.NA).dropna()
    horas_raw = horas_raw.replace("", pd.NA).dropna()

    raw_matches = sorted(set(leads_raw).intersection(set(horas_raw)))
    print("\n[JOIN] Coincidencias exactas (sin normalizar):", len(raw_matches))
    if len(raw_matches):
        print("  Muestra:", raw_matches[:20])

    leads_norm = leads_raw.map(_normalize_for_compare).replace("", pd.NA).
↪dropna()
    horas_norm = horas_raw.map(_normalize_for_compare).replace("", pd.NA).
↪dropna()
    norm_matches = sorted(set(leads_norm).intersection(set(horas_norm)))
    print("[JOIN] Coincidencias exactas (normalizadas):", len(norm_matches))
    if len(norm_matches):
        print("  Muestra:", norm_matches[:20])

    return {
        "leads_nonblank": int(len(leads_raw)),
        "horas_nonblank": int(len(horas_raw)),
        "leads_unique_raw": int(leads_raw.nunique()),
        "horas_unique_raw": int(horas_raw.nunique()),
        "matches_raw": int(len(raw_matches)),
        "leads_unique_norm": int(leads_norm.nunique()),
        "horas_unique_norm": int(horas_norm.nunique()),
        "matches_norm": int(len(norm_matches)),
    }

print("--- Perfilado LEADS ---")
leads_profile = _profile_table(df_leads)
display(leads_profile)
save_df_csv(leads_profile, OUTPUT_DIR / "profiling_leads_columns.csv")

print("\n--- Perfilado HORAS ---")
horas_profile = _profile_table(df_horas)
display(horas_profile)
save_df_csv(horas_profile, OUTPUT_DIR / "profiling_horas_columns.csv")

# Calidad de las llaves candidatas para join
_company_quality(df_leads, "Company", "LEADS")
_company_quality(df_horas, "EMPRESA", "HORAS")

# Evidencia cuantitativa: ¿hay intersección exacta?
match_summary = _exact_match_summary()
if match_summary:

```

```
save_df_csv(pd.DataFrame([match_summary]), OUTPUT_DIR /
↳"profiling_join_company_match_summary.csv")
```

--- Perfilado LEADS ---

	columna	dtype	n_null	pct_null	n_unique
0	First Name	float64	440	100.000000	0
1	Last Name	float64	440	100.000000	0
2	Email	float64	440	100.000000	0
3	Phone	float64	440	100.000000	0
4	Mobile	float64	440	100.000000	0
5	Website	float64	440	100.000000	0
6	No. of Employees	float64	440	100.000000	0
7	Annual Revenue	float64	440	100.000000	0
8	Linkedin	float64	440	100.000000	0
9	País.	str	415	94.318182	8
10	Lead Source	str	408	92.727273	1
11	Industry	str	408	92.727273	4
12	Cargo	str	87	19.772727	290
13	Interés	str	0	0.000000	1
14	Company	str	0	0.000000	330

[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestio
n_enrichment\outputs\profiling_leads_columns.csv
shape: (15, 5)

--- Perfilado HORAS ---

	columna	dtype	n_null	pct_null	n_unique
0	AVANCE_REAL	float64	412	100.000000	0
1	AVANCE_ESTIMADO	float64	412	100.000000	0
2	ID_COL_RESPONSABLE	float64	379	91.990291	7
3	FACTURACION	float64	235	57.038835	120
4	HORAS_ESTIMADAS	float64	187	45.388350	107
5	HORAS_EJECUTADAS_FACTURABLES	float64	26	6.310680	314
6	HORAS_EJECUTADAS	float64	20	4.854369	319
7	FECHA_CORTE	datetime64[us]	19	4.611650	5
8	EN_EJECUCION	float64	4	0.970874	2
9	MOSTRAR_LISTAS	int64	0	0.000000	2
10	LUGAR_POR_DEFECTO	int64	0	0.000000	2
11	TIPO_ALCANCE	str	0	0.000000	3
12	OCUPACION	str	0	0.000000	4
13	ID_ESP	int64	0	0.000000	15
14	EMPRESA	str	0	0.000000	120
15	ID_EMP	int64	0	0.000000	120
16	NOMBRE	str	0	0.000000	372
17	ID_PRO	int64	0	0.000000	412

[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestio
n_enrichment\outputs\profiling_horas_columns.csv

shape: (18, 5)

[LEADS] Calidad de Company:

- filas: 440
- n_null: 0
- n_blank (null o vacío): 0
- n_unique (no-null): 330
- n_duplicadas (por valor no-null): 110

	Company	freq
0	GRUPO ALEN	10
1	PEPSICO	7
2	GRUPO BIMBO	7
3	WALMART	7
4	BACARDI	5
5	LAMOSA	5
6	CASA CUERVO	5
7	SEGUROS MONTERREY NEW YORK LIFE	5
8	UNILEVER	4
9	AIG	4
10	NOVARTIS	4
11	ASTRAZENECA	3
12	SANOFI	3
13	RAPPI	3
14	GNP SEGUROS	3

- n_unique_norm: 328
- colisiones_por_norm (valores distintos que caen al mismo norm): 112

[HORAS] Calidad de EMPRESA:

- filas: 412
- n_null: 0
- n_blank (null o vacío): 0
- n_unique (no-null): 120
- n_duplicadas (por valor no-null): 292

	EMPRESA	freq
0	Corporacion GPF	66
1	Corporación Maresa	22
2	FPA	21
3	Veolia Latam	21
4	Aseguradora del Sur	20
5	Intaco	12
6	La Fabril	10
7	NOVA Ecuador	10
8	Telefónica EC	10
9	Millicom - Telefónica PA, NI	9
10	Adium	8
11	Seguros del Pichincha	7

```

12             Veolia China      7
13             Induglob         6
14             Veolia Argentina  6

- n_unique_norm: 119
- colisiones_por_norm (valores distintos que caen al mismo norm): 293

[JOIN] Coincidencias exactas (sin normalizar): 0
[JOIN] Coincidencias exactas (normalizadas): 0
[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestio
n_enrichment\outputs\profiling_join_company_match_summary.csv
      shape: (1, 8)

```

1.10 5. Validación de hipótesis de integración

1.10.1 Hipótesis

Se plantea que:

Las empresas presentes en el dataset de leads deberían coincidir con las empresas presentes en el dataset de horas trabajadas.

1.10.2 Objetivo

Validar si es posible realizar un join confiable entre ambas fuentes.

1.11 6. Evaluación detallada (coincidencias exactas normalizadas y aproximadas)

El resumen de la sección 4.b (Perfilado rápido) puede ampliarse con una rutina más detallada de normalización y similitud. Esta sección se conserva porque aporta evidencia técnica adicional:

- normaliza nombres con eliminación de acentos,
- reduce ruido por sufijos legales comunes,
- calcula coincidencias exactas después de normalizar,
- y genera sugerencias por similitud para evidenciar que no existen matches confiables.

Esto respalda formalmente la decisión de **no forzar un join** entre ambas fuentes usando solo el nombre de la empresa.

```

[8]: import re
import unicodedata
import pandas as pd

def _strip_accents(s: str) -> str:
    return "".join(ch for ch in unicodedata.normalize("NFKD", s) if not
        unicodedata.combining(ch))

def _normalize_name(s: str) -> str:
    if s is None:
        return ""

```

```

s = str(s).strip()
s = _strip_accents(s)
s = s.upper()
# quitar caracteres raros, dejar letras/números/espacios
s = re.sub(r"[^A-Z0-9 ]+", " ", s)
# quitar sufijos legales comunes
s = re.sub(r"\b(SA|S A|S\.A|S\.A\.  

↪S|SAS|LTDA|CIA|CORP|INC|LLC|DE|DEL|LA|EL)\b", " ", s)
s = re.sub(r"\s+", " ", s).strip()
return s

# Tomar listas directamente desde los DataFrames (evita dependencia de
↪secciones eliminadas)
if "Company" not in df_leads.columns or "EMPRESA" not in df_horas.columns:
    print("No se puede comparar: falta Company o EMPRESA en los dataframes.")
else:
    companies = (
        df_leads["Company"]
        .astype("string")
        .fillna("")
        .map(lambda x: x.strip())
        .replace("", pd.NA)
        .dropna()
        .unique()
        .tolist()
    )
    empresas = (
        df_horas["EMPRESA"]
        .astype("string")
        .fillna("")
        .map(lambda x: x.strip())
        .replace("", pd.NA)
        .dropna()
        .unique()
        .tolist()
    )

    if not companies or not empresas:
        print("No hay valores suficientes para comparar (listas vacías).")
    else:
        comp_norm = {c: _normalize_name(c) for c in companies}
        emp_norm = {e: _normalize_name(e) for e in empresas}

        # 1) Coincidencias exactas después de normalizar
        inv_emp = {}
        for e, ne in emp_norm.items():
            inv_emp.setdefault(ne, []).append(e)

```

```

exact_matches = []
for c, nc in comp_norm.items():
    for e in inv_emp.get(nc, []):
        exact_matches.append({
            "Company": c,
            "EMPRESA": e,
            "tipo": "exact_norm",
            "score": 100,
        })

# Esquema estable incluso si no hay coincidencias
df_exact = pd.DataFrame(exact_matches, columns=["Company", "EMPRESA",
↪ "tipo", "score"])
print(f"Coincidencias EXACTAS (tras normalizar): {len(df_exact)}")
if len(df_exact):
    display(df_exact.sort_values(["Company", "EMPRESA"]).
↪ reset_index(drop=True))

# 2) Coincidencias por similitud (fuzzy). Si está rapidfuzz, mejor; si
↪ no, usa difflib.
rows = []
try:
    from rapidfuzz import process, fuzz

    scorer = fuzz.token_set_ratio
    emp_choices = list(emp_norm.keys())
    for c, nc in comp_norm.items():
        best = process.extractOne(nc, emp_choices, scorer=scorer)
        if best is None:
            continue
        best_norm, score, _ = best
        e_orig = inv_emp.get(best_norm, [None])[0]
        rows.append({
            "Company": c,
            "Company_norm": nc,
            "EMPRESA_best": e_orig,
            "EMPRESA_norm": best_norm,
            "score": float(score),
        })
    engine = "rapidfuzz"
except Exception:
    from difflib import SequenceMatcher

    def ratio(a, b):
        return SequenceMatcher(None, a, b).ratio() * 100

```

```

emp_choices = list(emp_norm.keys())
for c, nc in comp_norm.items():
    best_norm = None
    best_score = -1
    for en in emp_choices:
        sc = ratio(nc, en)
        if sc > best_score:
            best_score = sc
            best_norm = en
    e_orig = inv_emp.get(best_norm, [None])[0] if best_norm is not None else None
    rows.append({
        "Company": c,
        "Company_norm": nc,
        "EMPRESA_best": e_orig,
        "EMPRESA_norm": best_norm,
        "score": float(best_score),
    })
engine = "difflib"

df_fuzzy = pd.DataFrame(rows).sort_values("score", ascending=False).
reset_index(drop=True)
print(f"\nMotor de similitud: {engine}")
print("Sugerencia: revisar coincidencias con score alto (p.ej. >= 80).")

display(df_fuzzy.head(30))
umbral = 80
df_high = df_fuzzy[df_fuzzy["score"] >= umbral]
print(f"\nCoincidencias sugeridas con score >= {umbral}:")
print(len(df_high))
display(df_high.reset_index(drop=True))

# --- Persistencia de evidencia a outputs/ ---
try:
    save_df_csv(df_exact, OUTPUT_DIR / "profiling_exact_matches_norm.
    csv")
    save_df_csv(df_fuzzy.head(200), OUTPUT_DIR /
    "profiling_fuzzy_best_matches_top200.csv")
    save_df_csv(
        df_high.reset_index(drop=True),
        OUTPUT_DIR / f"profiling_fuzzy_matches_score_ge_{umbral}.csv",
    )
except Exception as e:
    print(f"[WARN] No se pudieron guardar artefactos de matching: {e}")

```

Coincidencias EXACTAS (tras normalizar): 0

Motor de similitud: rapidfuzz

Sugerencia: revisar coincidencias con score alto (p.ej. ≥ 80).

	Company	Company_norm	EMPRESA_best	EMPRESA_norm	score
0	OSRAM	OSRAM	CORSAM	CORSAM	72.727273
1	FEMSA	FEMSA	FADESA	FADESA	72.727273
2	SMI	SMI	BMI	BMI	66.666667
3	BIC	BIC	BAC	BAC	66.666667
4	UCB	UCB	UPC	UPC	66.666667
5	CBC	CBC	BAC	BAC	66.666667
6	Chronos	CHRONOS	CONSEP	CONSEP	61.538462
7	LACOSTE	LACOSTE	CONSEP	CONSEP	61.538462
8	BACARDI	BACARDI	BANRED	BANRED	61.538462
9	BACHOCO	BACHOCO	BAC	BAC	60.000000
10	PEPSICO	PEPSICO	SIC	SIC	60.000000
11	Abbvie	ABBVIE	AVIS	AVIS	60.000000
12	Onapis	ONAPIS	AVIS	AVIS	60.000000
13	BALL	BALL	BAC	BAC	57.142857
14	PROESSA	PROESSA	PROPHAR	PROPHAR	57.142857
15	PPG	PPG	KPMG	KPMG	57.142857
16	BECLE SAB	BECLE SAB	CEPSA	CEPSA	57.142857
17	AIG	AIG	AVIS	AVIS	57.142857
18	URBI	URBI	BMI	BMI	57.142857
19	Pozuelo	POZUELO	PUCE	PUCE	54.545455
20	ONTEX	ONTEX	CONSEP	CONSEP	54.545455
21	TRANE	TRANE	BANRED	BANRED	54.545455
22	ESACERO S.A.	ESACERO	ESPE	ESPE	54.545455
23	Parauco	PARAUCO	PUCE	PUCE	54.545455
24	Arcor	ARCOR	CORSAM	CORSAM	54.545455
25	COMEX	COMEX	CONSEP	CONSEP	54.545455
26	ICONN	ICONN	CONSEP	CONSEP	54.545455
27	Confiteca	CONFITECA	CONSEP	CONSEP	53.333333
28	CXO Forum	CXO FORUM	CORSAM	CORSAM	53.333333
29	EUROFARMA	EUROFARMA	PROPHAR	PROPHAR	50.000000

Coincidencias sugeridas con score ≥ 80 : 0

Empty DataFrame

Columns: [Company, Company_norm, EMPRESA_best, EMPRESA_norm, score]

Index: []

[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs\profiling_exact_matches_norm.csv

shape: (0, 4)

[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs\profiling_fuzzy_best_matches_top200.csv

shape: (200, 5)

[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs\profiling_fuzzy_matches_score_ge_80.csv

shape: (0, 5)

1.11.1 Interpretación técnica de esta validación

Si esta sección produce:

- **0 coincidencias exactas tras normalización**, y
- **0 coincidencias con score alto**,

entonces existe evidencia suficiente para afirmar que ambas fuentes no pueden integrarse de forma confiable mediante el nombre de la empresa.

Este resultado no representa una falla del código, sino un hallazgo del análisis de calidad e integración de datos.

1.12 7. Resultado de la validación

No se encontraron coincidencias exactas entre las empresas de ambas fuentes.

1.12.1 Interpretación

Esto indica que:

- Las fuentes no están alineadas
- No es posible realizar un join directo confiable
- Existe inconsistencia en la representación de entidades

1.12.2 Conclusión

Se rechaza la hipótesis de integración directa.

1.13 8. Próximos pasos (según evidencia)

- No forzar join Company EMPRESA por nombre: el profiling muestra que no hay coincidencias confiables.
- Preparar staging normalizado para BD/joins posteriores en 02_data_cleaning/01_raw_normalizado_export.ipynb (export a CSV).
- Definir estrategia alternativa de integración (p.ej. catálogo unificado de empresas / matching asistido / llaves adicionales).

```
[9]: # --- Verificación de outputs generados ---
expected_outputs = [
    OUTPUT_DIR / "profiling_leads_columns.csv",
    OUTPUT_DIR / "profiling_horas_columns.csv",
    OUTPUT_DIR / "profiling_join_company_match_summary.csv",
    OUTPUT_DIR / "profiling_exact_matches_norm.csv",
    OUTPUT_DIR / "profiling_fuzzy_best_matches_top200.csv",
    OUTPUT_DIR / "profiling_fuzzy_matches_score_ge_80.csv",
]

print("\n[CHECK] Outputs esperados en:", OUTPUT_DIR.resolve())
```

```

missing = []
for p in expected_outputs:
    if p.exists():
        print(" - OK      ", p.resolve())
    else:
        print(" - MISSING", p.resolve())
        missing.append(p)

if missing:
    raise FileNotFoundError(
        "Faltan outputs esperados. Revisa si ejecutaste todas las secciones del_
↪notebook.\n"
        + "\n".join([str(p.resolve()) for p in missing])
    )

```

[CHECK] Outputs esperados en: E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs

- OK E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs\profiling_leads_columns.csv
- OK E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs\profiling_horas_columns.csv
- OK E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs\profiling_join_company_match_summary.csv
- OK E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs\profiling_exact_matches_norm.csv
- OK E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs\profiling_fuzzy_best_matches_top200.csv
- OK E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs\profiling_fuzzy_matches_score_ge_80.csv